

Implementasi Dinamis Server Side Dalam Aplikasi Manajemen Kost

Server-side rendering digunakan untuk data fetching, authentication, dan operations yang butuh keamanan tinggi.

1. Dynamic Routing dengan Database Query

File: src/app/dashboard-admin/manage-rooms/page.tsx (Server Component)

Implementasi:

```
export default async function ManageRoomsPage({
  searchParams,
}): {
  searchParams: { kost_id?: string };
}) {
  const supabase = createServerComponentClient({ cookies });
  const { data: { user } } = await supabase.auth.getUser();

  if (!user) {
    return redirect('/login/admin');
  }

  // Dynamic query: fetch costs milik owner
  const { data: costs } = await supabase
    .from('costs')
    .select('id, name, address')
    .eq('owner_id', user.id)
    .order('name', { ascending: true });

  // Dynamic flow berdasarkan data
  if (!costs || costs.length === 0) {
    // Render empty state
    return <EmptyStateComponent />;
  }

  // Multiple costs & no selection → Show selection UI
  if (costs.length > 1 && !searchParams.kost_id) {
    return (
      <div className="grid grid-cols-3 gap-6">
        {costs.map((kost) => (
          <Link key={kost.id} href={` /dashboard-admin/manage-rooms?kost_id=${kost.id}`}>
            <h3>{kost.name}</h3>
            <p>{kost.address}</p>
          </Link>
        ))}
      </div>
    );
  }
}
```

```

    );
  }

  // Get selected kost & fetch rooms
  const selectedKostId = searchParams.kost_id || costs[0].id;
  const kost = costs.find(k => k.id === selectedKostId) || costs[0];

  const { data: rooms } = await supabase
    .from('rooms')
    .select('*')
    .eq('kost_id', kost.id)
    .order('room_number');

  // Render room list
  return (
    <div>
      <h1>{kost.name}</h1>
      <p>{kost.address}</p>

      <div className="grid grid-cols-3 gap-6">
        {rooms?.map((room) => (
          <RoomCard key={room.id} room={room} />
        ))}
      </div>
    </div>
  );
}

```

Penjelasan:

- **Server Component:** fetch data di server sebelum render
- **Dynamic routing:** menggunakan searchParams untuk filter
- **Conditional rendering** berdasarkan:
 - User authentication
 - Jumlah costs (0, 1, atau >1)
 - Selected kost
- Query database secara dynamic berdasarkan owner_id dan kost_id
- Zero client-side JavaScript untuk initial render

Dynamic Behavior:

- URL: /manage-rooms → tampilkan semua atau selection
 - URL: /manage-rooms?kost_id=xxx → tampilkan rooms untuk kost tertentu
 - Auto-redirect jika tidak authenticated
 - Empty state jika tidak punya properties
-

2. Dynamic Breadcrumbs dengan UUID Resolution

File: src/app/dashboard-admin/layout.tsx

Implementasi:

```
export default function DashboardAdminLayout({ children }: { children:
React.ReactNode }) {
  const pathname = usePathname();
  const [breadcrumbs, setBreadcrumbs] = useState<Array<{name: string, href:
string}>>>([]);

  useEffect(() => {
    const generateBreadcrumbs = async () => {
      const pathSegments = pathname.split('/').filter(segment =>
segment && segment !== 'dashboard-admin');
      const newBreadcrumbs = [{ name: 'Dashboard', href: '/dashboard-
admin' }];

      for (let i = 0; i < pathSegments.length; i++) {
        const segment = pathSegments[i];
        const href = '/dashboard-admin/' + pathSegments.slice(0, i +
1).join('/');
        let name = '';

        // Check if segment is UUID
        const isUUID = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-
f]{4}-[0-9a-f]{12}$/i.test(segment);

        if (isUUID) {
          if (pathSegments[i - 1] === 'manage-rooms') {
            // Resolve UUID ke nama kost
            const { data: kostData } = await supabase
              .from('kots')
              .select('name')
              .eq('id', segment)
              .single();
            name = kostData?.name || segment;
          }
          else if (pathSegments[i - 1] === 'manage') {
            // Resolve UUID ke nomor kamar
            const { data: roomData } = await supabase
              .from('rooms')
              .select('room_number')
              .eq('id', segment)
              .single();
            name = `Kamar ${roomData?.room_number}` || segment;
          }
        } else {
          // Find in navigation items

```

```

        href);
        const navItem = navigation.find(item => item.href ===
        name = navItem ? navItem.name : segment.replace('-', '
');
    }
    newBreadcrumbs.push({ name, href });
  }
  setBreadcrumbs(newBreadcrumbs);
};

generateBreadcrumbs();
}, [pathname, supabase]);

return (
  <div>
    { /* Breadcrumbs */ }
    <nav className="flex items-center space-x-2">
      {breadcrumbs.map((item, index) => (
        <div key={item.href} className="flex items-center space-
x-2">
          {index > 0 && <ChevronRight className="w-4 h-4" />}
          {index === breadcrumbs.length - 1 ? (
            <span className="font-semibold text-emerald-600">
              {item.name}
            </span>
          ) : (
            <Link href={item.href}>{item.name}</Link>
          )}
        </div>
      )}}
    </nav>
    {children}
  </div>
);
}

```

Penjelasan:

- **Dynamic breadcrumbs** dari URL pathname
- **UUID resolution:** query database untuk convert UUID ke human-readable name
- Pattern matching untuk detect UUID format
- Conditional queries berdasarkan parent segment:
 - /manage-rooms/{kostId} → query costs table
 - /manage/{roomId} → query rooms table
- Fallback ke segment name jika tidak match

Dynamic Behavior:

- URL: /dashboard-admin/manage-rooms/abc-123/manage/def-456
 - Breadcrumbs: Dashboard > Manage Rooms > **Kost Melati** > Manage > **Kamar A101**
 - Real database names, bukan UUID
 - Active breadcrumb (last item) dengan warna berbeda
-

3. Server Actions dengan Cascade Operations

File: src/app/dashboard-admin/manage-rooms/[kostId]/manage/[roomId]/page.tsx

Implementasi:

```
<form
  action={async (formData) => {
    "use server";

    const supabase = createServerComponentClient({ cookies });
    const supabaseAdmin = createClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );

    const isAvailable = formData.get("is_available") === "on";

    try {
      // Dynamic cascade delete jika room berubah ke "tersedia"
      if (isAvailable && !room.is_available) {
        console.log('Room status berubah: terisi → tersedia');

        // 1. Delete booking_requests
        const { error: deleteBookingError } = await supabaseAdmin
          .from("booking_requests")
          .delete()
          .eq("room_id", params.roomId);

        if (deleteBookingError && deleteBookingError.code !==
'PGRST116') {
          throw new Error('Gagal menghapus booking requests');
        }

        // 2. Delete payments
        await supabaseAdmin
          .from("payments")
          .delete()
          .eq("room_id", params.roomId);

        // 3. Delete penyewa
        const { error: deleteError, count } = await supabaseAdmin
          .from("penyewa")
          .delete()
          .eq("room_id", params.roomId)
          .eq("kost_id", params.kostId);

        if (deleteError) {
          throw new Error('Gagal menghapus penyewa');
        }
      }
    }
  }
```

```

        console.log(`Berhasil menghapus ${count || 0} penyewa`);
    }
    // Update room data
    const { error: updateError } = await supabase
        .from("rooms")
        .update({
            room_number: formData.get("room_number"),
            price: Number(formData.get("price")),
            facilities: formData.get("facilities"),
            is_available: isAvailable,
        })
        .eq("id", params.roomId);

    if (updateError) {
        throw new Error('Gagal mengupdate room');
    }

    console.log('Room berhasil diupdate');

} catch (error) {
    console.error('Error:', error);
}

redirect(`/dashboard-admin/manage-rooms`);
}}
>
    { /* Form fields */ }
</form>

```

Penjelasan:

- **Inline Server Action:** defined langsung di form action
- **Conditional cascade operations:** hanya jika isAvailable berubah dari false → true
- **Sequential deletes:** booking → payments → penyewa (urutan penting!)
- **Error handling:** check error code PGRST116 (no rows found, bukan error)
- **Supabase Admin Client:** bypass RLS untuk admin operations
- **Logging:** console.log untuk tracking operations
- **Redirect:** programmatic navigation setelah success

Dynamic Behavior:

- Jika toggle availability OFF → ON: trigger cascade delete
 - Jika toggle ON → OFF: langsung update tanpa delete
 - Count deleted records dan log ke console
 - Redirect ke manage-rooms setelah selesai
-

4. Dynamic Booking Approval with Multiple Database Operations

File: src/app/dashboard-admin/booking-request/actions.ts

Implementasi:

```
export async function approveBooking(formData: FormData) {
  const supabase = createServerActionClient({ cookies });
  const requestId = formData.get('requestId') as string;

  const { data: { user } } = await supabase.auth.getUser();
  if (!user) {
    throw new Error('Anda tidak memiliki izin.');
  }

  // 1. Fetch booking request data
  const { data: bookingRequest, error: fetchError } = await supabase
    .from('booking_requests')
    .select('*')
    .eq('id', requestId)
    .single();

  if (fetchError || !bookingRequest) {
    throw new Error('Booking request tidak ditemukan');
  }

  // 2. Validasi room availability (dynamic check)
  const { data: room } = await supabase
    .from('rooms')
    .select('is_available')
    .eq('id', bookingRequest.room_id)
    .single();

  if (room && !room.is_available) {
    throw new Error('Kamar sudah terisi oleh penghuni lain');
  }

  // 3. Update payment status (only for this specific user & room)
  const { error: paymentUpdateError } = await supabase
    .from("payments")
    .update({ status: 'approved' })
    .eq("room_id", bookingRequest.room_id)
    .eq("penyewa_id", bookingRequest.user_id)
    .eq("status", "pending");

  if (paymentUpdateError) {
    console.error('Payment update error:', paymentUpdateError);
  }

  // 4. Update booking status
```



```

const { error: updateError } = await supabase
  .from('booking_requests')
  .update({ status: 'approved' })
  .eq('id', requestId);

if (updateError) {
  throw new Error(`Gagal update booking: ${updateError.message}`);
}

// 5. Delete old tenant (jika ada) - IMPORTANT!
await supabase
  .from('penyewa')
  .delete()
  .eq('room_id', bookingRequest.room_id);

// 6. Insert new tenant
const dueDate = formData.get('dueDate') as string;
const { error: insertError } = await supabase
  .from('penyewa')
  .insert({
    penyewa_id: bookingRequest.user_id,
    kost_id: bookingRequest.kost_id,
    room_id: bookingRequest.room_id,
    due_date: dueDate,
  });

if (insertError) {
  throw new Error(`Gagal menambah penyewa: ${insertError.message}`);
}

// 7. Update room availability
const { error: roomError } = await supabase
  .from('rooms')
  .update({ is_available: false })
  .eq('id', bookingRequest.room_id);

if (roomError) {
  throw new Error(`Gagal update room: ${roomError.message}`);
}

// 8. Delete approved booking request
await supabase
  .from('booking_requests')
  .delete()
  .eq('id', requestId);

// 9. Auto-reject other bookings for same room
await supabase
  .from('booking_requests')

```

```
.delete()  
.eq('room_id', bookingRequest.room_id);  
  
revalidatePath('/dashboard-admin/booking-request');  
revalidatePath('/dashboard-admin/rooms');  
}
```

Penjelasan:

- **Complex server-side workflow** dengan 9 sequential operations
- **Dynamic validation:** check room availability sebelum approve
- **Atomic-like operations:** multiple database updates dalam sequence
- **Error handling** di setiap step dengan descriptive messages
- **Data consistency:** delete old tenant sebelum insert new
- **Auto-cleanup:** reject booking requests lain untuk room yang sama
- **Path revalidation:** update cache untuk multiple pages

Dynamic Behavior:

- Stop execution jika room sudah terisi (validation)
 - Only update payment dengan match exact user & room
 - Delete conflicting bookings automatically
 - Revalidate affected routes untuk instant UI update
-

5. API Route dengan Conditional Logic

File: src/api/validate-referral/route.ts

Implementasi:

```
export async function POST(request: Request) {
  const supabase = createRouteHandlerClient({ cookies });

  // Service role client untuk bypass RLS
  const supabaseAdmin = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!
  );

  // Check authentication
  const { data: { user }, error: authError } = await
supabase.auth.getUser();

  if (authError || !user) {
    return NextResponse.json(
      { error: 'Unauthorized' },
      { status: 401 }
    );
  }

  const { referralCode } = await request.json();

  if (!referralCode || typeof referralCode !== 'string') {
    return NextResponse.json(
      { error: 'Kode referral tidak valid' },
      { status: 400 }
    );
  }

  // DEBUG: Get all profiles dengan referral code (for development)
  const { data: allProfiles } = await supabaseAdmin
    .from('profiles')
    .select('id, email, role, referral_code')
    .not('referral_code', 'is', null);

  // Dynamic query: Find owner dengan referral code
  const { data: owner, error: ownerError } = await supabaseAdmin
    .from('profiles')
    .select('id, email, role, referral_code')
    .eq('referral_code', referralCode.trim().toUpperCase())
    .eq('role', 'pemilik')
    .single();

  if (ownerError || !owner) {
```

```

// DEBUG: Return detailed error untuk development
return NextResponse.json(
  {
    error: 'Kode referral tidak ditemukan atau tidak valid',
    debug: {
      searchedCode: referralCode.trim().toUpperCase(),
      availableCodes: allProfiles?.map(p => ({
        code: p.referral_code,
        role: p.role,
        email: p.email
      })),
      errorDetail: ownerError?.message
    }
  },
  { status: 404 }
);
}

// Update user profile dengan referral code
const { error: updateError } = await supabaseAdmin
  .from('profiles')
  .update({
    used_referral_code: referralCode.trim().toUpperCase(),
    referral_verified: true,
  })
  .eq('id', user.id);

if (updateError) {
  return NextResponse.json(
    { error: 'Gagal menyimpan kode referral' },
    { status: 500 }
  );
}

// Dynamic fetch: Get costs owned by this owner
const { data: costs, error: costsError } = await supabaseAdmin
  .from('costs')
  .select('id, name, address')
  .eq('owner_id', owner.id);

if (costsError) {
  console.error('Error fetching costs:', costsError);
}

return NextResponse.json({
  success: true,
  message: 'Kode referral berhasil diverifikasi',
  owner: {
    id: owner.id,
  }
});

```

```
        email: owner.email,  
      },  
      costsCount: costs?.length || 0,  
    });  
  }
```

Penjelasan:

- **API Route Handler** dengan multiple response types
- **Authentication check:** return 401 jika tidak login
- **Input validation:** type checking untuk referralCode
- **Case-insensitive search:** .trim().toUpperCase()
- **Debug mode:** return available codes untuk development
- **Service role client:** bypass RLS untuk cross-user operations
- **Dynamic response:** include costs count dari owner

Dynamic Behavior:

- Return different status codes: 401, 400, 404, 500, 200
 - Debug info hanya di development (available codes list)
 - Update user profile hanya jika referral valid
 - Fetch additional data (costs) untuk context
-

6. Cron Job API untuk Automatic Cleanup

File: src/api/check-expired-bookings/route.ts

Implementasi:

```
const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);

export async function GET() {
  try {
    // Dynamic query: Find expired bookings
    const { data: expiredBookings, error: fetchError } = await supabaseAdmin
      .from('booking_requests')
      .select('id, room_id')
      .eq('status', 'pending')
      .lt('booking_expires_at', new Date().toISOString())
      .is('paid_at', null);

    if (fetchError) {
      return NextResponse.json({ error: 'Failed to fetch' }, { status: 500 });
    }

    if (!expiredBookings || expiredBookings.length === 0) {
      return NextResponse.json({
        message: 'No expired bookings found',
        count: 0
      });
    }

    // Batch update: Set booking status to expired
    const bookingIds = expiredBookings.map(b => b.id);
    const { error: updateBookingError } = await supabaseAdmin
      .from('booking_requests')
      .update({ status: 'expired' })
      .in('id', bookingIds);

    if (updateBookingError) {
      return NextResponse.json({ error: 'Failed to update bookings' }, {
        status: 500 });
    }

    // Batch update: Reset room availability
    const roomIds = expiredBookings.map(b => b.room_id);
    const { error: updateRoomError } = await supabaseAdmin
      .from('rooms')
      .update({ is_available: true });
  }
}
```

```

        .in('id', roomIds);

    if (updateRoomError) {
        return NextResponse.json({ error: 'Failed to update rooms' }, { status:
500 });
    }
    return NextResponse.json({
        success: true,
        message: `${expiredBookings.length} expired booking(s) processed`,
        count: expiredBookings.length,
        bookings: expiredBookings
    });

} catch (error) {
    return NextResponse.json({ error: 'Internal server error' }, { status:
500 });
}
}

// POST method dengan authorization untuk cron security
export async function POST(request: Request) {
    const authHeader = request.headers.get('authorization');

    // Dynamic validation: Check secret key
    if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
        return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    return GET();
}

```

Penjelasan:

- **Cron Job Endpoint:** designed untuk scheduled execution
- **Dynamic time-based query:** compare dengan current timestamp
- **Batch operations:** update multiple records sekaligus dengan `.in()`
- **Service role client:** full database access untuk cleanup
- **Dual methods:** GET (testing) dan POST (production dengan auth)
- **Authorization:** Bearer token dengan secret key
- **Detailed response:** count dan list of processed bookings

Dynamic Behavior:

- Query berdasarkan current server time
 - Process bookings yang expired dalam batch
 - Return count untuk monitoring
 - Skip jika tidak ada expired bookings
 - Secure POST endpoint untuk production cron
-

7. Dynamic Content Based on User Role & State

File: src/app/dashboard-penyewa/page.tsx (Server Component)

Implementasi:

```
const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);

export default async function TenantDashboardPage() {
  const supabase = createServerComponentClient({ cookies });
  const { data: { user } } = await supabase.auth.getUser();

  if (!user) {
    redirect('/login/user');
  }

  // Dynamic query: Check if user is active tenant
  const { data: tenant, error: tenantError } = await supabaseAdmin
    .from('penyewa')
    .select('id, due_date, room_id, kost_id')
    .eq('penyewa_id', user.id)
    .maybeSingle();

  if (!tenant) {
    // User bukan penyewa → redirect ke dashboard user biasa
    redirect('/dashboard-user');
  }

  // Fetch related data
  const { data: room } = await supabaseAdmin
    .from('rooms')
    .select('room_number, price, facilities')
    .eq('id', tenant.room_id)
    .maybeSingle();

  const { data: kost } = await supabaseAdmin
    .from('kosti')
    .select('name, address')
    .eq('id', tenant.kost_id)
    .maybeSingle();

  if (!room || !kost) {
    return <ErrorComponent />;
  }

  // Dynamic calculation: Days until due date
  const dueDate = new Date(tenant.due_date);
```



```

const today = new Date();
today.setHours(0, 0, 0, 0);
dueDate.setHours(0, 0, 0, 0);
const diffTime = dueDate.getTime() - today.getTime();
const daysUntilDue = Math.ceil(diffTime / (1000 * 60 * 60 * 24));

const isNearDue = daysUntilDue <= 7 && daysUntilDue > 0;
const isOverdue = daysUntilDue < 0;

return (
  <div>
    {/* Dynamic Status Card - Color based on due date */}
    <div className={`rounded-2xl p-6 ${
      isOverdue
        ? 'bg-red-50 border-2 border-red-200'
        : isNearDue
        ? 'bg-yellow-50 border-2 border-yellow-200'
        : 'bg-emerald-50 border-2 border-emerald-200'
      }`}>
      <h3 className="text-lg font-bold">
        {isOverdue ? 'Pembayaran Terlambat' : isNearDue ? 'Segera Jatuh
Tempo' : 'Status Aktif'}
      </h3>
      <p className={isOverdue ? 'text-red-600' : 'text-emerald-600'}>
        {isOverdue
          ? `Terlambat ${Math.abs(daysUntilDue)} hari`
          : `${daysUntilDue} hari lagi`
        }
      </p>
    </div>

    {/* Dynamic Alert Message */}
    {(isOverdue || isNearDue) && (
      <div className={`mt-4 p-3 rounded-lg ${
        isOverdue ? 'bg-red-100' : 'bg-yellow-100'
      }`}>
        <p className={isOverdue ? 'text-red-700' : 'text-yellow-700'}>
          {isOverdue
            ? 'Harap segera lakukan pembayaran untuk menghindari denda atau
pemutusan kontrak.'
            : 'Jatuh tempo pembayaran sudah dekat. Segera lakukan
pembayaran.'
          }
        </p>
      </div>
    )}

    {/* Info Cards */}
    <div className="grid md:grid-cols-2 gap-6">

```

```

<div className="bg-white rounded-2xl shadow-lg p-6">
  <h3>Informasi Kost</h3>
  <p>Nama: {kost.name}</p>
  <p>Kamar: {room.room_number}</p>
  <p>Fasilitas: {room.facilities}</p>
</div>

<div className="bg-white rounded-2xl shadow-lg p-6">
  <h3>Informasi Pembayaran</h3>
  <p>Biaya: Rp {room.price.toLocaleString('id-ID')}</p>
  <p>Jatuh Tempo: {new Date(tenant.due_date).toLocaleDateString('id-ID')}</p>
  <p className={isOverdue ? 'text-red-600' : 'text-emerald-600'}>
    Sisa: {isOverdue ? `Terlambat ${Math.abs(daysUntilDue)} hari` :
`${daysUntilDue} hari`}
  </p>
</div>
</div>
);
}

```

Penjelasan:

- **Server Component** dengan complex business logic
- **Role-based routing:** redirect based on user state (tenant vs non-tenant)
- **Service role client:** bypass RLS untuk read tenant data
- **Dynamic calculations:** days until due date di server
- **Conditional styling:** 3 states dengan warna berbeda (green/yellow/red)
- **Conditional rendering:** alert hanya muncul jika near due atau overdue
- **Date manipulation:** server-side date calculations

Dynamic Behavior:

- Route protection: auto-redirect jika bukan tenant
 - 3 status states berdasarkan daysUntilDue:
 - Normal (>7 hari): Green theme
 - Near due (≤7 hari): Yellow theme dengan warning
 - Overdue (<0 hari): Red theme dengan urgent alert
 - Alert message berubah berdasarkan status
 - Empty state jika data room/kost tidak ditemukan
-

8. File Upload dengan Storage Integration

File: src/app/dashboard-admin/payment-method/actions.ts

Implementasi:

```
export async function addPaymentMethod(state: ServerActionState, formData:
FormData) {
  const supabase = createServerActionClient({ cookies });
  const type = formData.get("type") as string;
  const kostId = formData.get("kost_id") as string;

  const { data: { user } } = await supabase.auth.getUser();
  if (!user) {
    return { success: false, message: 'Anda tidak memiliki izin.' };
  }

  try {
    // Validation: Check ownership
    const { data: kostData, error: kostError } = await supabase
      .from("ksts")
      .select("id")
      .eq("id", kostId)
      .eq("owner_id", user.id)
      .limit(1);

    if (kostError || !kostData || kostData.length === 0) {
      throw new Error("Anda tidak memiliki izin untuk mengelola kost ini.");
    }

    if (type === "rekening") {
      // Simple insert untuk rekening
      const bankName = formData.get("bank_name") as string;
      const accountNumber = formData.get("account_number") as string;

      const { error } = await supabase.from("payment_methods").insert({
        kost_id: kostId,
        type,
        bank_name: bankName,
        account_number: accountNumber,
      });

      if (error) throw new Error(`Gagal menambahkan rekening:
      ${error.message}`);
    }
    else if (type === "qris") {
      const file = formData.get("qris_image") as File;

      if (!file || file.size === 0) {
        throw new Error("Gambar QRIS harus diunggah.");
      }
    }
  }
}
```

```

}

// Dynamic filename dengan UUID
const fileExtension = file.name.split(".").pop();
const fileName = `${uuidv4()}.${fileExtension}`;
const filePath = `qris/${kostId}/${fileName}`;

// Upload ke Supabase Storage
const { error: uploadError } = await supabase.storage
  .from("payment_methods")
  .upload(filePath, file, {
    cacheControl: "3600",
    upsert: false,
  });

if (uploadError) {
  throw new Error(`Gagal mengunggah QRIS: ${uploadError.message}`);
}

// Get public URL
const { data: { publicUrl } } = supabase.storage
  .from("payment_methods")
  .getPublicUrl(filePath);

// Insert dengan URL
const { error } = await supabase.from("payment_methods").insert({
  kost_id: kostId,
  type,
  qris_url: publicUrl,
});

if (error) {
  throw new Error(`Gagal menyimpan URL QRIS: ${error.message}`);
}
}

revalidatePath("/dashboard-admin/payment-methods");
return {
  success: true,
  message: "Metode pembayaran berhasil ditambahkan!",
};
} catch (e: any) {
  return { success: false, message: e.message };
}
}

```

Penjelasan:

- **Server Action** dengan file upload capability

- **Conditional logic:** berbeda untuk rekening vs QRIS
- **File validation:** check file exists dan not empty
- **UUID filename:** generate unique name untuk avoid conflicts
- **Organized storage:** path structure `qris/{kostId}/{uuid}.{ext}`
- **Storage settings:** `cacheControl`, `upsert false`
- **Public URL generation:** get URL setelah upload
- **Database insert:** save URL reference, bukan file binary
- **Path revalidation:** update cache setelah insert

Dynamic Behavior:

- Type 'rekening': simple insert (no file)
 - Type 'qris': multi-step (validate → upload → get URL → insert)
 - Dynamic path structure berdasarkan `kostId`
 - Error handling di setiap step
 - Return success/error state untuk UI feedback
-

9. Conditional Delete with Dependency Checks

File: src/app/dashboard-admin/manage-rooms/actions.ts

Implementasi:

```
export async function deleteRoom(roomId: string) {
  try {
    // Use admin client untuk full access
    const { error } = await supabaseAdmin
      .from("rooms")
      .delete()
      .eq("id", roomId);

    if (error) {
      console.error("Error deleting room:", error);
      return {
        success: false,
        error: error.message,
      };
    }

    // Revalidate untuk update UI
    revalidatePath("/dashboard-admin/manage-rooms");

    return {
      success: true,
      message: "Kamar berhasil dihapus",
    };
  } catch (error) {
    console.error("Unexpected error:", error);
    return {
      success: false,
      error: "Terjadi kesalahan saat menghapus kamar",
    };
  }
}
```

Note: Cascade deletes handled di page component, bukan di action:

```
// In page.tsx server action (inline)
action={async () => {
  "use server";

  const supabaseAdmin = createClient(...);

  // Sequential deletes dalam urutan yang benar
  // 1. booking_requests
  await supabaseAdmin.from("booking_requests").delete().eq("room_id",
roomId);
```

```
// 2. bookings
await supabaseAdmin.from("bookings").delete().eq("room_id", roomId);

// 3. payments
await supabaseAdmin.from("payments").delete().eq("room_id", roomId);

// 4. penyewa
await supabaseAdmin.from("penyewa").delete().eq("room_id", roomId);

// 5. room itself
await supabaseAdmin.from("rooms").delete().eq("id", roomId);

redirect("/dashboard-admin/manage-rooms");
}}
```

Penjelasan:

- **Separation of concerns:** action untuk simple delete, page untuk complex cascade
- **Admin client:** bypass RLS untuk delete operations
- **Error code checking:** PGRST116 = no rows (bukan error)
- **Sequential execution:** delete dependencies sebelum main record
- **Programmatic redirect:** navigate after completion
- **Path revalidation:** clear Next.js cache

Dynamic Behavior:

- Check dependencies existence (might not exist)
 - Graceful handling untuk missing records
 - Return success/error untuk UI feedback
 - Redirect hanya setelah semua deletes berhasil
-